I gang med JavaScript

Up and running

Hello World 👍✋✌ - 3 små opgaver

Nu da vores udviklingsmiljø virker skal vi i gang med at lære det at kende.

1. Vores Hello World!
   Aktiverer en dialogboks
2. Hello Console!
   Vi benytter browserens værktøj
3. Hello HTML
   Vi skriver i viewporten

```
alert("Hello World!");
```

```
console.log("Hello Console!");
```

```
document.querySelector("h1").textContent = "Hello HTML!";
```

** Bemærk at den sidste version kræver, at der er et h1-element i html-dokumentet*

3 forskellige output

```
alert("Hello World!");
```

JavaScript



Output i browseren: En dialogboks

Vi kommer ikke til at bruge dette igen.

```
console.log("Hello Console!");
```

JavaScript



Output i browserens Console (en del af Web Developer Tools)

Web Developer Tools er et sæt af hjælpeværktøjer i browseren.

Vi har indtil videre mest brugt Inspectoren (til HTML og CSS)

Fra nu af bruger vi også **Konsollen**.

Åben Developer Tools med ⌥ + ⌘ + i

Åben Konsollen med esc

```
<body>
   <h1></h1>
   <script src="js/hello.js"></script>
</body>
```

HTML

```
document.querySelector("h1").textContent = "Hello HTML!";
```

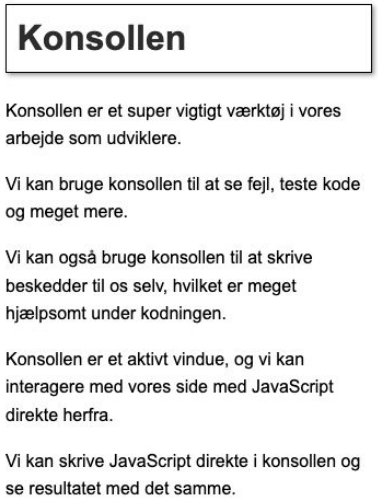
JavaScript

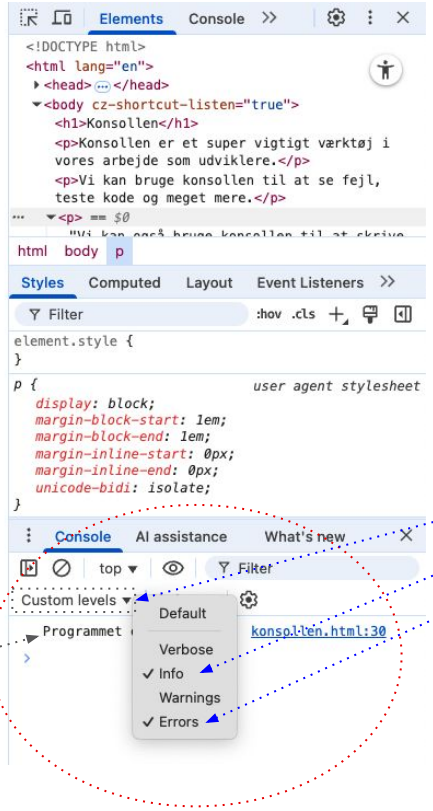
Hello HTML!

Output i viewporten: <h1> teksten er ændret

Konsollen (eng. *Console*) - et uundværligt værktøj

Viewporten





The Console
Kan aktiveres/deaktiveres med **[esc]**

Her er en besked skrevet ud samt hvilken linje i dokumentet, der gjorde det

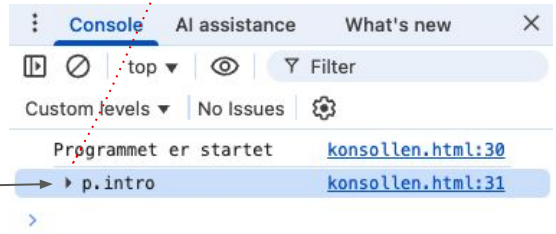
Husk at aktivere "Info" og "Errors"

JavaScript DOM

Vi kan altså finde ethvert element i det renderede HTML med JavaScript metoden: **document.querySelector();**

Lad os prøve at skrive det ud i konsollen:

```
console.dir(document.querySelector("p.intro"));
```





Her har jeg valgt en **<p>** med en **class="intro"**
Med **console.dir** skriver jeg elementet ud i konsollen.

Når vi folder elementet ud, kan vi se, at browseren har rigtig mange informationer om elementet.

JavaScript DOM

textContent er en værdi, som vi kan sætte.
Det er en af de mange værdier som er gemt i browserens hukommelse, når den har hentet vores website. Når vi bruger **textContent = *"noget tekst"*** på et element, overskriver vi hvadend, der stod i elementet i forvejen.

Vi kan også manipulere elementet direkte

Fx kan vi ændre elementets tekstindhold:

```
document.querySelector("p.outro").textContent = "Denne tekst står nu inde i vores paragraf";

document.querySelector("h1").textContent = "NY OVERSKRIFT i H1";


document.querySelector("p.outro").textContent += "Denne tekst står nu OGSÅ inden i vores paragraf";
```

Eller

```
document.querySelector("p.outro").parentNode.innerHTML += "<p class='obs'>Vi har nu tilføjet en paragraf</p>";
```

I gang med **første** sæt af øvelser

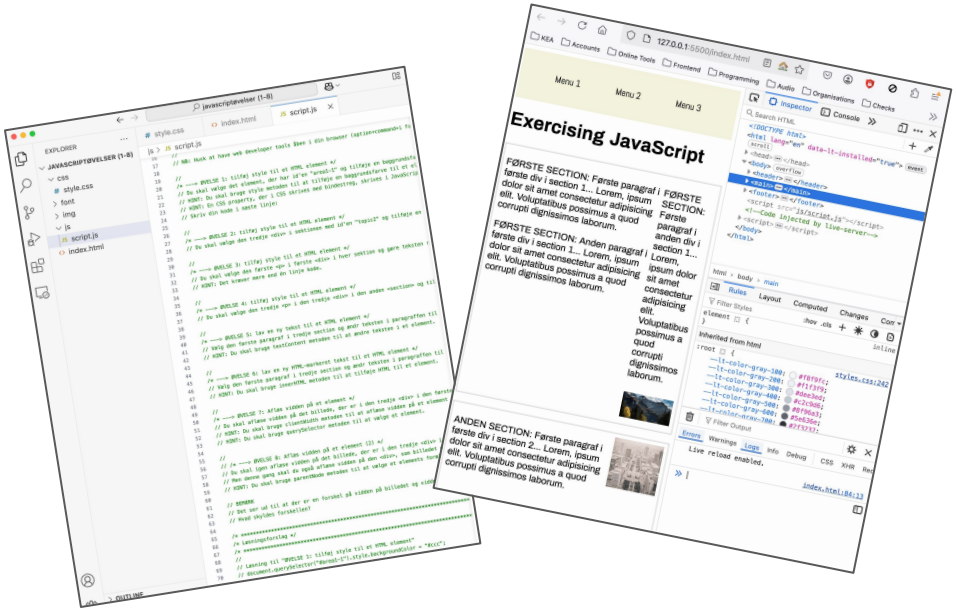
> Download zip-filen "Javascriptøvelser (1-8).zip" fra It's Learning

> Placer filen i din øvelsesmappe

> Udpak filen

> Åben mappen i VS Code

> Arbejd i javascript filen



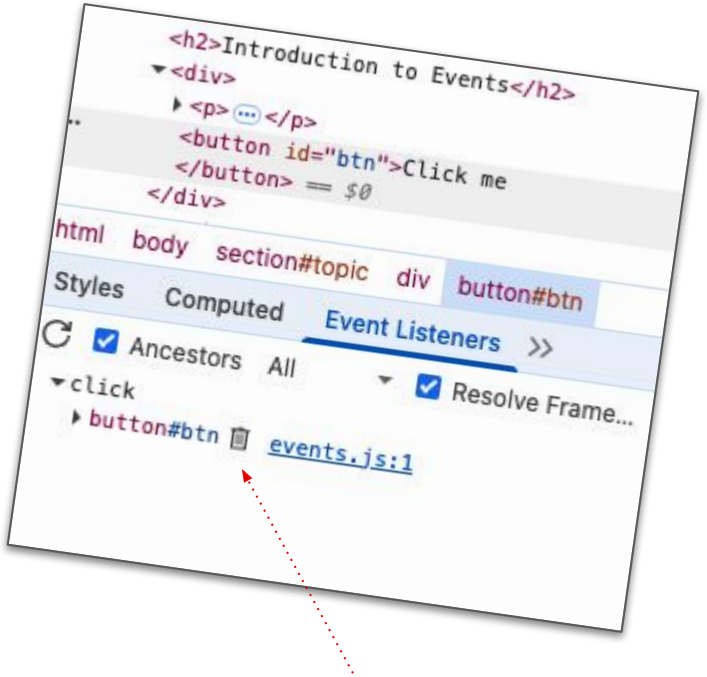
Lav funktioner og tilføj eventListeners

Vi skal i gang med at gøre siden interaktiv
Dvs. at sidens indhold ændrer sig alt efter brugerens aktiviteter.

Vi har allerede set lidt af den slags med pseudo class **:hover**

Med JavaScript kan vi gøre alle elementer interaktive og ændre mange ting.

Vi kan ændre det element vi aktiverer
- men vi kan også ændre andre elementer



Hvis vi klikker på elementer kan vi se, at der er et 'click' event på det.

Vi skal bruge det, der hedder en

eventListener

eventListener anatomi

Her er en eventListener.
Lad os se hvad denne javascript kodelinje består af

```
document.querySelector("#btn").addEventListener("click", clickBtn);
```

Når man bruger en *metode*, skal der være *parentes*.

Inde i parentesen kan man definere nogle *parametre*, som adskilles med komma.

Det renderede HTML dokument (DOM)

Den metode vi bruger til at vælge et element

Elementet vi vil bruge udvalgt med en CSS selector

En ny metode. Det er med denne metode, at vi sætter en **event** på elementet

Der er mange **typer** af eventListeners. Her har vi valgt et klik-event.

Event-typen skal skrives i anførsestegn og skal hedde det rigtige, officielle navn.
Se online reference

Eventets **target**.
Dvs. den funktion, der skal invokes, når eventet aktiveres.

eventListener og funktion

```
17 document.querySelector("#btn").addEventListener("click", clickBtn);
```

Navnet på funktionen
Finder vi selv på.

En eventListener's **target** er en *funktion*.

Funktionen *invokes* ikke når eventListeners **defineres** (*her: linje 17*) - først når den bliver aktiveres af fx brugeren (*som ved "click"*).

Men skal skal vi også have lavet funktionen …

```
function clickBtn() {
 console.log(`Klikket`);
}
```

Vi skal bruge en

function()

function anatomi

Vi kigger nærmere på koden for funktionen

Vi bestemmer selv, hvad funktionen skal hedde.

Men ingen mellemrum!

Med et JavaScript **keyword** definerer vi funktionen

```
function clickBtn() {
 console.log(`Klikket`);
 document.querySelector("h1").textContent = "hej";
}
```

Efter funktionsnavnet skal vi skrive et krøllede parenteser.

Inde i funktionen kan vi skrive al den JavaScript, vi har lyst til.

Koden bliver ikke afviklet, før funktionen *invokes*.

Hvad sker der her?



```
document.querySelector("#btn").addEventListener("click", newWidth);


function newWidth() {
 console.log(`newWidth`);
 document.querySelector("img").style.rotate = "10deg";
 document.querySelector("img").style.transition = "rotate 1s";
}
```

Snak parvist om hvad der står i koden og noter ned, hvad I kan aflæse, at der sker
- så detaljeret som muligt

I gang med **andet** sæt af øvelser

> Download zip-filen "Javascriptøvelser (9-15).zip" fra It's Learning

> Placer filen i din øvelsesmappe

> Udpak filen

> Åben mappen i VS Code

> Arbejd i javascript filen

